CITS5501/CITS3501 Project 2026 Phase 1

Version: 0.1
Date: 10 August, 2025

1. Project admin

1.1. Project timeline

- This project contributes **35**% towards your final grade this semester.
- The project is marked out of **50**, and has
  - an individual component (5 marks, due 11:59 pm, Tues 18 August). For this task, we ask you to reflect on previous group projects you have participated in, and what challenges may arise in this one. A separate post will be made in the discussion forum once the questions and submission point are opened.
  - a group component (45 marks, due dates specified below).
- The group component has 3 deliverables:
  - phase 1 (planning; 5 marks, due 11:59 pm, Thurs 20 August). In this phase you will make contact with your group, and put infrastructure and plans in place for phase 2.
  - phase 2 (implementation and testing; 25 marks, due 11:59 pm, Thurs 1 October). In this phase you will implement an application and a test suite for it.
  - phase 3 (review; 15 marks, due 11:59 pm, Thurs 15 October). In this phase, you will swap your implementation with another, randomly-selected group, and will be tasked with reviewing their test suite and how it contributes to project quality.

In semester week 8, commencing Monday 14 September you will need to formally check in with your facilitator and show your progress on phase 2. This is not worth marks, but is required before you can submit phase 2.

2. Project rules

Expectations and guidance on the use of AI tools, group formation, project clarifications and marking rubrics, late submission, and academic conduct are provided at CITS5501/3501 FAQs and CITS5501/3501 Group Assessment FAQs. For this project you are expected to be familiar with this guidance and comply with the rules.

You may use implementations of algorithms from any published source, but must credit that source in your code.

2.1. Project groups

Students will be allocated to a group of approximately 5 students for this project. Each group is drawn from the same timetabled lab class, and is assigned a lab facilitator. Groups should meet with their facilitator about once a fortnight during your normal lab time. The facilitator will support you for team forming and project matters and will typically be the marker for your project submissions.

The group list is posted in the Blackboard LMS discussion forum, and is also available on Moodle.

As explained in the Group Assessment FAQs, individual contributions are taken into account. All group members will be required to complete a Feedback Fruits peer evaluations shortly after the final (phase 3) project submission.

Once these peer evaluations are done, each group member will receive a Group Contribution Factor (GCF). The final individual mark for phases 1–3 is $raw_mark \times GCF$. The minimum possible GCF is 0, and the maximum GCF is 1.2 (but no student can be awarded more than 100% of the marks available for any phase).

2.2. Clarifications and changes to the project specification

You are encouraged to start reading through this project brief and planning your work as soon as it is released. Any queries regarding the project should be posted to Blackboard LMS discussion forum. Any clarifications or amendments that need to be made will be posted by teaching staff in the forum.

2.3. Format and submission of your work

All code and reports should be submitted by making a Moodle submission.

- Please structure your answers using numbered headings where appropriate.
- If you include any diagrams, charts or tables, they must be clear, legible and large enough to read when viewed on-screen at 100% magnification.
- If you give scholarly references, you may use any standard citation style you wish, as long as it is consistent. However, a recommended citation style is "AMS short alpha-numeric" (see AMS style guide, sec 10.3).
- Diagrams, charts, tables, bibliographies and reference lists do not count towards any word-count maximums.

Submitted code should meet the usual guidelines for CITS5501/3501 code:

- code should be clearly written, well-formatted, and easy for others to understand
- function bodies should not contain excessive inline comments
- code should compile without errors or warnings
- code should follow sound programming practices, including:
  - the use of meaningful comments
  - well chosen identifier names
  - appropriate choice of basic data-structures, data-types, and functions
  - appropriate choice of control-flow constructs
  - proper error-checking of any library functions called, and
  - cleaning up/closing any files or resources used.

Any public methods (including JUnit `@Test` methods) that you write should include a documentation block. (If using Java, follow the Oracle guidelines for Writing Doc Comments. If using Python, you should follow PEP 257/Google style for your docstrings.)

Note that code which fails to compile or does not pass type-checking phases will be awarded (at most) a very small number of marks. Markers are not obliged to fix your code if it does not compile or fails to type-check.

Your code should never emit any output to standard output unless specified in the project brief, but may emit log messages to standard error.

2.4. Documenting assumptions

If, after posting questions to the discussion and having received answers, you believe there is still insufficient information for you to complete part of this project, you should document any *reasonable assumptions* you had to make in order to answer a question or write a portion of code.

Make sure you give each assumption a number (e.g. A001, A002, etc) and explain why you think it's reasonable.

Then in your code or elsewhere, you can briefly refer to these assumptions (e.g. "This test case assumes that Assumption A001 holds, so that we can …").

An assumption which contradicts anything in the project specificatio or postings made by staff in the discussion forum is *prima facie* not reasonable.

2.5. Marking environment

The phase 1 and 3 deliverables do not require you to submit code. For phase 2, your application will be implemented in either Python or Java.

You may assume that your code will be tested on an Ubuntu 22.04 environment (x86-64 processor), with the following packages installed:

- `python3`
- `uv`
- `openjdk-21-jdk`

3. Phase 1

3.1. Background: Scraping an Online Policy Library

Your task is to build a command-line program that can scrape an online policy library and retrieve policy documents in a range of formats, including PDF, Word and RTF, and allow end users to query the retrieved documents. The catch is that the Web is not always a cooperative or predictable environment: the server may be temporarily unavailable, may try to limit how frequently your program accesses it, and individual pages or documents may contain broken links, unexpected responses, or malformed markup. Your group will need to decide what to do when things don't go according to plan – and your testing should give you confidence that those decisions actually work. You may implement your scraper in either **Java or Python**. Before you get the detailed specification or see the site, you'll first need to think about how you would approach the problem, what could go wrong, and how you might test a solution.

3.2. Tasks for project phase 1 [5 marks]

3.2.1. Task 1: Team organisation

You should agree on the following details, and record them (in writing) in a Collaboration & Workflow Plan:

- **Team Communication & Responsibilities**
  - How will the group communicate throughout the project? What will be the frequency of meetings (e.g., weekly; fortnightly; as needed) and the meeting format (e.g., face-to-face; online via video

calls or chat platforms)? Are there any preferred communication tools (e.g., Discord, Slack, MS Teams, email)?
  - How will the group ensure accountability and track progress? Will you assign any initial roles?
- **Version Control Strategy**
  - Specify where the project's documents will be hosted (e.g., GitHub, GitLab, Bitbucket).
  - How will you handle merging members' contributions? If you're familiar with version control branching strategies, will you adopt any particular strategy or workflow? (e.g., feature branches, main/dev workflow)
  - Identify any version control policies (e.g., commit message conventions, review/approval process before merging).
- **Resource Risk Management**
  - Outline potential resourcing risks (e.g. member illness) to the project and how they will be mitigated.
- **Optional – group name**
  - Nominate a name for your group – less than 20 characters, may be multiple words. Pronounceable, workplace friendly, and no embedded HTML tags or SQL commands, please. ☺

3.3. Task 2: Group Git repository and task tracking

Create a **private** Git repository for the members of your group (you will later need to add your facilitator, for review and marking), hosted on a service such as GitHub, GitLab, or BitBucket.

All members of the group must be able to commit to the repository. Your group should maintain your code, report and other work in version control using the repository.

You will also need a way of tracking task allocation and completion – this could be a shared spreadsheet (either in the repository, or on MS Teams or Google Docs), a Kanban board, or something else.

Neither the repository nor tracker are submitted for assessment, but teaching staff will ask to see them to ensure they're in place, and may ask to review them later (for instance, to clarify contributions made by group members).

Tips:

- When storing documents or reports in version control, you're generally best using a plain text format (such as Markdown, LaTeX, CSV, RTF, or flat Office formats). Your final report, however, should be a PDF generated from that plain text format.
- Version control systems do a poor job of tracking changes to binary formats such as `.pptx` (PowerPoint), `.xlsx` (Excel), or PDF.
- In general, best practice is to keep in version control the source text or code your binary artifacts are *derived* from, but not the binary artifacts themselves. (Exception: you may want to store a binary copy of the final submitted PDF of your report.)

3.4. Task 3: Report

Create a phase 1 report Markdown file in your repository. This (and a PDF generated from it) will be the main deliverable for phase 1. We will provide a sample Makefile that could be used for generating a PDF from your Markdown file.

In your report, answer the questions below.

Planning and organization

1. Where is your group's Collaboration & Workflow Plan maintained? Is it in the Git repository, or elsewhere?
2. What tool or approach are you using to track assigned tasks and completion? Why did you choose it? Has anyone in the group used it before?

Language and tooling

For this project, you may choose to implement your application in either Java or Python.

3. Which language has your group chosen? Have any members of the group not used it – and if so, how will they get familiar with it?

What factors influenced your choice of implementation language? Rank the top three in order of importance.

– Familiarity with the language
– Team members' experience
– Library support
– Ease of debugging
– Performance
– Maintainability
– Assessment considerations
– Career value
– Other (explain)

What advantages do you expect your chosen language to give your team? What disadvantages or risks do you expect? (Every choice has costs.) If you had to implement this project in the other language, what do you think would be most difficult? What might be easier?

4. What build/dependency management tool will you use (e.g. `uv` or `poetry` for Python; Gradle or Maven for Java; something else)? Has anyone in the group used it before – or will this be the first time setting up a project with it?

Technical risks

5. What parts of a web scraper do you expect could most easily fail, and how?
6. Which external factors could make your software stop working after it has been completed?
7. Which components of your project do you think will be easiest to test? Which do you expect will be hardest?
8. What assumptions is your software likely to make about the website it scrapes? How could those assumptions be violated?

Team practices

9. Suppose a serious bug is discovered two days before the deadline. How would your team investigate and respond?
10. What will convince your team that a feature is "finished"?

11. If two group members disagree about whether software is "working correctly", how will you resolve that disagreement?

Code quality

12. Python groups: Will you use a type checker (e.g. `mypy` or `pyright`), and if so, will it run in CI or just periodically/manually? Has anyone used one before? Do you plant to use a linter (e.g. `pylint`, `black`)?
    Java groups: Is your team familiar with any static analysis/linting tools (e.g. Checkstyle, SpotBugs, PMD)? Do you plan to use any?
13. Suppose a group member's contribution doesn't pass your linting or type checking checks. What will be the consequences? Will merges be blocked if these checks fail, or will the checks be advisory only?

Testing framework

14. What test framework will you use (e.g. `pytest` for Python; JUnit 5 for Java; something else)? Has the group used it before, or only heard of it?

CI / automation

15. Will you set up continuous integration (e.g. GitHub Actions) to run your test suite automatically, or run tests manually before submission? Have any members of your group used continuous integration before?

Testing mindset

16. Suppose your scraper is finished and submitted. Now imagine another student is trying to break your software (get it to exhibit failures or unexpected behaviour). What might they try first?
17. How will you know that your software is producing the *correct* output, rather than merely *some* output?
18. What evidence would increase your confidence that your software is reliable?
19. At this stage of the project, what do you think will be the biggest challenge in producing reliable software?

Process and standards

20. What will be your code review process — does someone other than the author review changes before merging, or is this informal?

3.5. Phase 1 Submission

For Phase 1 you should submit the markdown and pdf versions of your report in the CITS5501/3510 Project submission area in Moodle.

One student only per group should submit the project.

The report should include links to your team's private github repository for the project.

At the start of your report, list all members of the group and their

– student number
– GitHub/GitLab/other user ID.

Please number the answers in your report. Remember to ensure your facilitator has access to your repository.